

Improving Scientific Reasoning in Vision-Language Models with Code Sandboxes, Agentic Correction, Learned Verification and Retrieval on mmJEE-Eval

Phase-2 Progress Report: Baselines, Pilot and Fixes

Arpit Kumar (12340350) Saurav Gupta (12341940)
Machine Learning course project, IIT Bhilai 9 October 2026

Abstract—mmJEE-Eval shows that open vision-language models (VLMs) are weak at numerical computation and at turning a detected error into a fix. We wrap a *frozen* 4-bit VLM in an inference-time framework with four modules: a code sandbox (program-of-thought), a Critic/Corrector loop that localises the first wrong step, a learned solution verifier (the only trained part) and retrieval of verified exemplars. This Phase-2 report covers the implementation, the baselines, an end-to-end pilot and the errors we found and fixed. Our harness reproduces the base paper’s Qwen2.5-VL-7B result (13.8% vs. 12.4% on the 2025 test set), and our substrate Qwen3-VL-8B reaches 43.4% Pass@1 with the paper’s prompts. In a 40-question pilot, the full framework reaches 57.5% vs. 29.5% for the baseline and 35.0% for compute-matched self-consistency. The pilot exposed 7 method-level problems (e.g. lost code state, blocked libraries, unparsed critic replies); we explain the cause and the fix for each, together with 11 bugs from a pre-run code review and 9 infrastructure issues.

I. INTRODUCTION AND PHASE-2 SCOPE

mmJEE-Eval [1] contains 1,460 bilingual (English/Hindi) JEE-Advanced questions from 2019–2025, given only as images. The paper reports low accuracy for open models (12.4% for Qwen2.5-VL-7B on 2025) and single-pass error-correction success of only 1.1–5.2%. It releases an evaluation harness but no method. Our SOP proposes a training-free framework around a frozen VLM, evaluated additively against a compute-matched self-consistency (SC) baseline [2].

This report (Phase 2) contains: (i) the implemented framework and evaluation harness (Sec. III); (ii) baselines: a reproduction of the paper with Qwen2.5-VL-7B and our substrate Qwen3-VL-8B on the full 2025 test set (Sec. IV); (iii) an end-to-end pilot of all four modules on 40 test questions (Sec. V); (iv) the errors found and how and why we fixed them (Sec. VI). The full-scale run of all modules on all 190 test questions is the subject of the final report (Sec. VII).

II. DATA AND EVALUATION PROTOCOL

Split. HF release ArkaMukherjee/mmJEE-Eval: train/dev = 2019–2024 (1,270 questions), test = 2025 (190 = 95 English + 95 Hindi twins; 84 Numerical, 46 MCQ-Single, 42 MCQ-Multiple, 18 Matching). The 102 questions of 2026 are excluded. Hyper-parameters, the verifier and the RAG threshold are selected on 2024 only.

Scoring. `extract_answer` and `is_answer_correct` are copied verbatim from the paper’s notebooks; a test checks the copies with `ast`. The public data lacks the `acceptable_values` column the scorer needs for range answers (e.g. “8.70 TO 9.10”), so we rebuild it on a 0.001 grid; all 691 numerical gold answers then score as correct. 12 MCQ-Single rows of 2023 with multi-letter golds are retyped MCQ-Multiple, and 4 duplicated question IDs get unique IDs.

Metrics. Pass@1 = mean accuracy over samples (one “run” per sample index, like the paper’s run average). 95% CIs use a cluster bootstrap over EN/HI twin pairs (2,000 resamples), and differences use the paired version. SC is compared at the *same generated-token budget*.

III. FRAMEWORK

Fig. 1 shows the pipeline. The substrate is Qwen3-VL-8B-Instruct [14] (AWQ 4-bit), served by vLLM [13] on one RTX 3060 (12 GB); InternVL3.5-8B [15] (AWQ) is the cross-model critic. Sampling: $T=0.7$, top- p 0.8, top- k 20, at most 4,096 new tokens per turn.

A. Code sandbox (PoT) [3], [4]. The model writes “Step k :” reasoning with `python` blocks. Generation stops at each closed block; the block runs in an isolated subprocess (AST whitelist of math/numpy/sympy/scipy; no file or network I/O; CPU, memory and output limits) and its output is appended before generation continues (at most 3 blocks). If no `\boxed{}` answer appears, the SOP’s plain-CoT fallback is used.

B. Agentic correction [5], [6]. A critic receives the image and the numbered steps and answers `FIRST_ERROR_STEP: k` or `NO_ERROR`. The corrector keeps steps $1..k-1$ and re-derives from step k (with the sandbox). The loop stops on `NO_ERROR`, an unchanged answer, or 3 rounds. We compare a same-model and a cross-model critic, since self-critique alone often fails [7].

C. Learned verifier [8], [9]. Training data: 8 PoT samples per training question, labelled automatically by the verbatim scorer (pilot: 150 questions, 1,200 solutions). Features: length, steps, code blocks run/failed, fallback, agreement with sibling answers, answer plausibility (format, magnitude), type/subject/language, plus a sentence embedding. Models: logistic regression and XGBoost; GroupKFold by twin pair

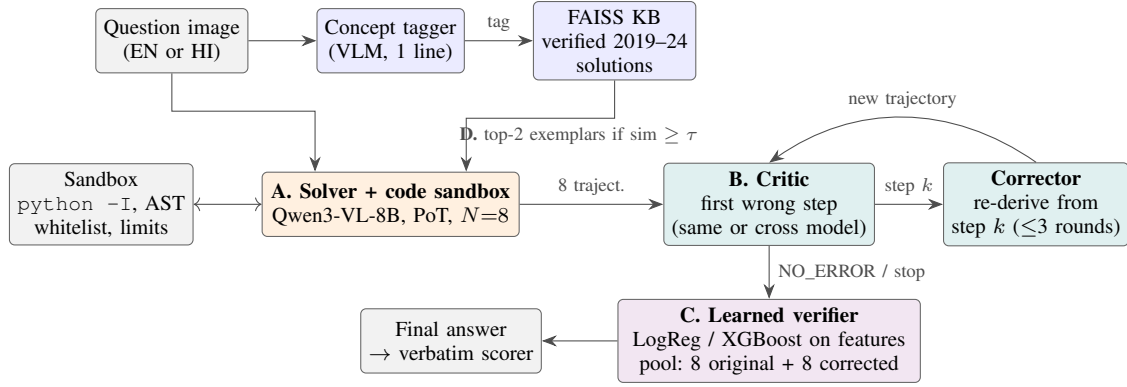


Fig. 1. Inference-time framework around a frozen VLM. Only the verifier (C) is trained, on auto-labelled solutions of 2019–2024 questions. The KB (D) contains no 2025 question and no near-duplicate of one.

TABLE I
BASELINES ON THE 2025 HELD-OUT SET WITH THE PAPER’S PROMPTS (SINGLE-PASS CoT). QWEN2.5-VL-7B REPRODUCES THE PAPER’S NUMBER; QWEN3-VL-8B IS OUR SUBSTRATE.

Metric (2025 test set)	Qwen2.5-VL-7B	Qwen3-VL-8B
Pass@1 (all 190)	13.8	43.4
95% CI (cluster bootstrap)	[10.7, 17.3]	[37.1, 49.9]
Std over runs	1.9 (10 runs)	1.9 (16 runs)
Paper (Table 3)	12.4 ± 1.9	–
Numerical	6.5	40.0
MCQ-Single	28.7	49.6
MCQ-Multiple	6.2	46.9
Matching	27.2	35.8
English	14.9	48.5
Hindi	12.6	38.4
Mathematics	12.0	55.1
Chemistry	19.5	41.7
Physics	9.7	33.2
Diagram not needed	14.0	48.5
Diagram needed	13.3	30.0
Self-consistency, 16 votes	–	53.7
Mean generated tokens / sample	–	2,705
Samples hitting the 4,096 cap	–	40%

on 2019–2023, dev = 2024. The selected verifier picks the final answer from the 16-candidate pool.

D. RAG [10]. Each question gets a one-line English concept tag (subject, topic, method) from the VLM; tags are embedded (bge-small [12]) and searched with FAISS [11] within the same subject. The top-2 entries above a similarity threshold τ (tuned on 2024 queries against a 2019–2023 KB) are inserted as method-only exemplars.

Engineering. Every model call and code execution is cached in SQLite, keyed by model, prompt, seed and settings, so every stage resumes after a crash and evaluation runs offline. `-pilot` runs write separately named outputs, so pilot and full results never mix.

IV. BASELINES

Reproduction of the paper (Qwen2.5-VL-7B). Before building on the harness we checked it against a published number. With the paper’s prompts (Appendix C), image first, and the verbatim scorer, Qwen2.5-VL-7B scores $13.8 \pm 1.9\%$ over

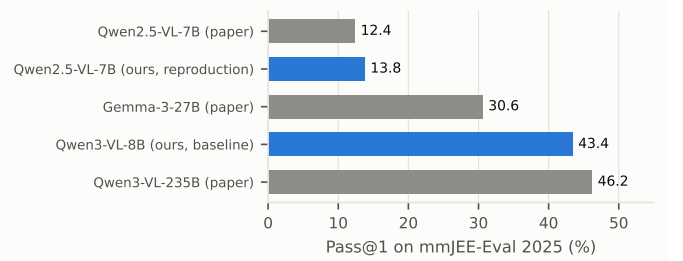


Fig. 2. Our baselines (blue) next to the base paper’s numbers (grey) on 2025.

10 runs (CI [10.7, 17.3]) vs. $12.4 \pm 1.9\%$ in the paper (Table I, Fig. 2). The paper’s value lies inside our CI, so prompts, image handling and scoring match the original protocol.

Substrate baseline (Qwen3-VL-8B). Same prompts, single-pass CoT, 16 samples per question on all 190 test questions: Pass@1 43.4% (CI [37.1, 49.9], run std 1.9). This is far above the paper’s 7B model and close to Qwen3-VL-235B (46.2%), a model-generation effect, since the harness reproduces the paper (above). The paper’s weaknesses persist (Fig. 3): Hindi trails English by 10 points, questions that need the diagram trail by 18 points, and Physics is the weakest subject. Majority voting over 16 samples reaches 53.7% (Fig. 4), which is the compute-hungry reference the framework has to beat. 40% of the samples hit the 4,096-token cap; we keep the same cap for all configurations. *Caveat:* Qwen3-VL was released after the 2025 exam, so pre-training exposure cannot be excluded; the paper’s contamination check is part of the final report.

V. PILOT OF THE FULL FRAMEWORK

Setup. The pilot runs every module end-to-end on 40 test questions (20 EN/HI twin pairs, stratified by question type) with a verifier and KB built from 150 training questions (8 PoT samples each). It was run before the fixes of Sec. VI (“v1”). The pilot questions are harder than average (baseline 29.5% vs. 43.4% on all 190).

Results (Table II, Figs. 5–6). Each module adds accuracy: the code sandbox lifts the baseline from 29.5% to 41.2%,

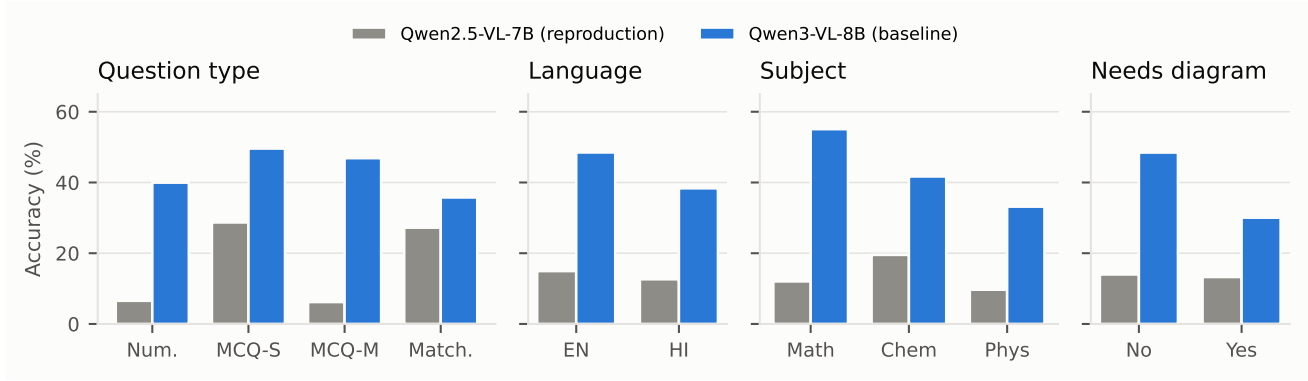


Fig. 3. Baseline accuracy by question type, language, subject and diagram dependence.

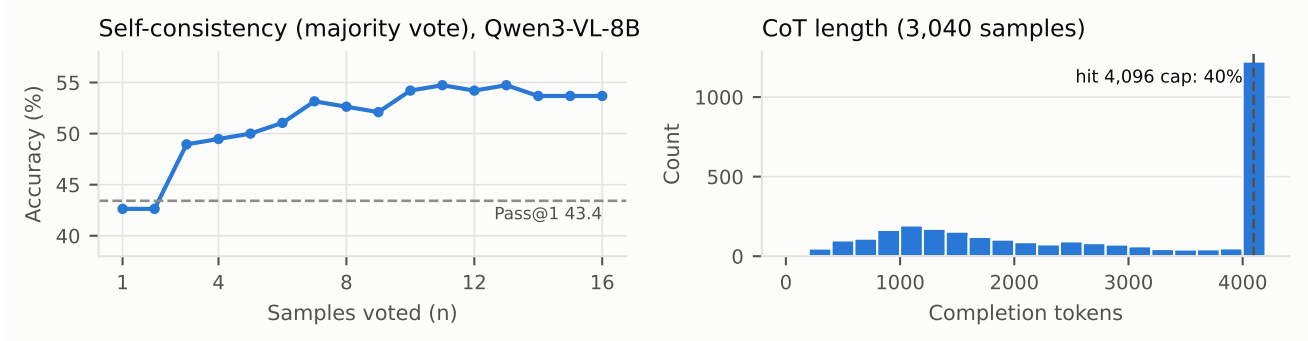


Fig. 4. Qwen3-VL-8B baseline. Left: majority-vote accuracy vs. number of samples (dashed: Pass@1). Right: completion length; the dashed line is the token cap.

TABLE II
END-TO-END PILOT (V1): 40 TEST QUESTIONS (20 EN/HI PAIRS, STRATIFIED BY TYPE), VERIFIER AND KB FROM 150 TRAINING QUESTIONS. FULL FRAMEWORK VS. MATCHED-COMPUTE SC: +22.5 POINTS, PAIRED 95% CI [+7.5, +37.5].

Configuration (additive)	Acc.	95% CI	Num.	Tok/q
Baseline (single-pass CoT)	29.5	[17.8, 41.6]	19.4	2.7k
Self-consistency (matched)	35.0	[20.0, 52.5]	33.3	43.1k
+ Code sandbox	41.2	[28.1, 54.4]	38.9	3.5k
+ Correction (same)	46.6	[32.8, 60.3]	43.1	7.7k
+ Correction (cross)	43.1	[29.7, 56.2]	43.1	5.5k
+ Learned verifier	50.0	[32.5, 67.5]	50.0	44.0k
+ Retrieval (RAG)	57.5	[40.0, 72.5]	55.6	46.6k

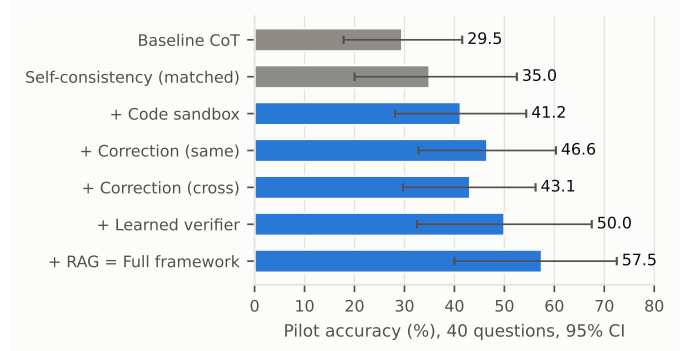


Fig. 5. Pilot (v1), additive rows with 95% CIs.

correction adds 2–5 points, the verifier and RAG bring the full framework to 57.5%. The full framework beats SC at the same token budget by +22.5 points (paired CI [+7.5, +37.5]). With 40 questions the CIs are about ± 15 points, so these numbers are preliminary; they show that the pipeline works end to end and where it fails.

Correction (Table III, Fig. 7). The step-localised, multi-round loop fixes 17–20% of the wrong solutions its critic detects, well above the paper’s single-pass 1.1–5.2%. The same-model critic flags almost everything (79% detection but 53% false alarms); the cross-model critic was hampered by unparsed replies (76), which motivated fix 3 below.

How the sandbox is used (Fig. 8). Only 27% of trajectories execute code, and they are the most accurate. 38% end in the CoT fallback, with the lowest accuracy; this motivated the analysis in fix 4.

Verifier (Table IV, Fig. 9). Correctness is predictable: AUC ≈ 0.87 on the 2024 dev year and 0.78–0.81 on the 2025 pilot pool with handcrafted features, which carry most of the signal; the sentence embedding alone is weak (0.57–0.70). With only 1,200 training solutions, selection is at most 2.5 points above majority voting and far below the oracle (75%), so the main headroom is in selection.

TABLE III
CORRECTION LOOP IN THE PILOT (V1, 320 CHAINS EACH). DETECT = CRITIC FLAGS AN ORIGINALLY WRONG SOLUTION; FALSE AL. = FLAGS A CORRECT ONE; FIX|DET. = DETECTED WRONG SOLUTIONS THAT END CORRECT.

Critic	Before → after	Detect	False al.	Fix det.	W→R / R→W	Unparsed
Same-model (Qwen3-VL)	41.2 → 46.6	79.3	53.0	16.8	25 / 8	0
Cross-model (InternVL3.5)	41.2 → 43.1	35.1	23.5	19.7	13 / 7	76
Full (RAG, cross)	42.8 → 46.2	40.4	21.2	20.3	15 / 4	76
Base paper (single pass)				1.1–5.2		

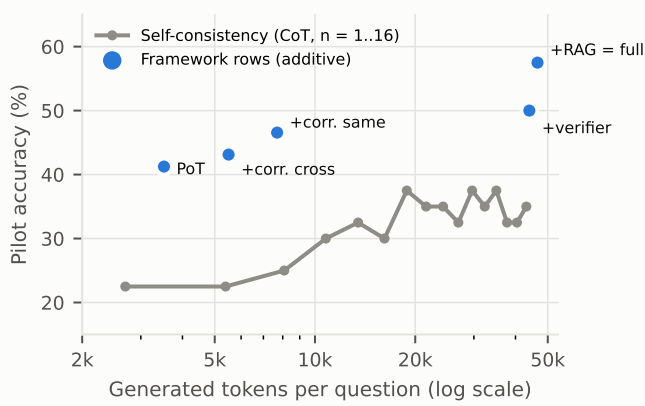


Fig. 6. Pilot accuracy vs. generated tokens per question. Grey: SC over n CoT samples. The framework rows lie above the SC curve at every budget.

TABLE IV
LEARNED VERIFIER IN THE PILOT (1,200 AUTO-LABELLED SOLUTIONS, 47.6% CORRECT). CV: GROUPKFOLD BY TWIN PAIR ON 2019–2023; DEV: 2024; TEST: THE 2025 PILOT POOL (16 CANDIDATES PER QUESTION). TEST SEL. = SELECTION ACCURACY (WEIGHTED VOTE).

Model / features	CV AUC	Dev AUC	F1	Test AUC	Test sel.
LR / handcrafted	0.858	0.867	0.736	0.781	55.0
LR / embedding	0.704	0.608	0.619	0.570	57.5
LR / all	0.863	0.876	0.743	0.794	57.5
XGB / handcrafted	0.855	0.874	0.754	0.794	55.0
XGB / embedding	0.622	0.591	0.588	0.603	55.0
XGB / all	0.847	0.870	0.785	0.807	55.0
Majority vote					55.0
Oracle (any correct)					75.0

RAG. The pilot KB has only 56 entries (only the 56 of the 75 pilot training pairs that had a verified-correct solution), and only 4 of the 40 questions retrieved an exemplar. The pilot therefore cannot attribute the last gain to retrieval; the final KB is built from all 1,270 training questions.

VI. ERRORS FOUND AND FIXES

We found problems in three ways: by reading the pilot traces (1,520 PoT trajectories = 320 test + 1,200 training, and 320 correction chains per critic), by a pre-run code review with three parallel reviewers, and while setting up the lab GPU. For each problem we report the evidence, why it matters and what we changed. Every code fix has a regression test.

Fixes from the pilot (Table V). We re-ran PoT and the cross-model correction on the same 40 questions with the fixes

(“v2”, Table VI, Fig. 10). The engineering failures are gone: no blocked or crashed code, and the critic returns a verdict for 500 of 501 calls. The critic now *detects* many more errors (56% vs. 35%), but its net effect on accuracy is unchanged within noise; turning detections into fixes remains the bottleneck, as in the paper. Fig. 11 shows the A/B test behind fix 4: forcing an answer from truncated reasoning is worse on three of four question types, so we kept the SOP’s fallback.

Code review and infrastructure (Tables VII–VIII). Before spending GPU time, three reviewers checked data/scoring/metrics, sandbox/PoT/agents and verifier/RAG/leakage in parallel and found 11 bugs; the most serious would have crashed the verifier (8), mismatched its train and test features (9) or made intervals too narrow (15). The lab issues were mostly memory limits of the 12 GB GPU and the shared PC’s out-of-memory daemon.

Testing. 73 pytest tests run on CPU with a mock VLM: scorer identity with the notebooks and all gold answers, extraction cases, sandbox limits (timeout, memory, blocked imports, output cap, replay, scipy), PoT and correction loops (fallback, verdict forcing, stop rules), verifier on synthetic data (AUC > 0.9) and CV indexing, RAG filters and threshold, leakage assertions (no 2025 twin in any training data or KB) and an end-to-end pipeline run.

VII. PLAN FOR THE FINAL REPORT

The full run applies the fixed (v2) pipeline to all 190 test questions and all 1,270 training questions on the lab GPU: CoT with 20 samples; PoT with 8 samples; same- and cross-model correction; PoT on the training set (verifier data and KB); RAG-PoT; full-framework correction; verifier training and best-of-16 selection; and the paper’s contamination check. The final report will contain the complete SOP Table I with matched-compute SC, significance tests, per-type/language/subject breakdowns, correction statistics compared with the paper, and the discussion.

VIII. LIMITATIONS

One substrate model and one test year (190 questions); the pilot has only 40 questions (CIs about ± 15 points); 4-bit weights and a 4,096-token budget (40% of baseline samples are truncated); the strict, unchanged scorer (no partial credit, first `\boxed{}` only); possible pre-training exposure of Qwen3-VL to the 2025 exam.

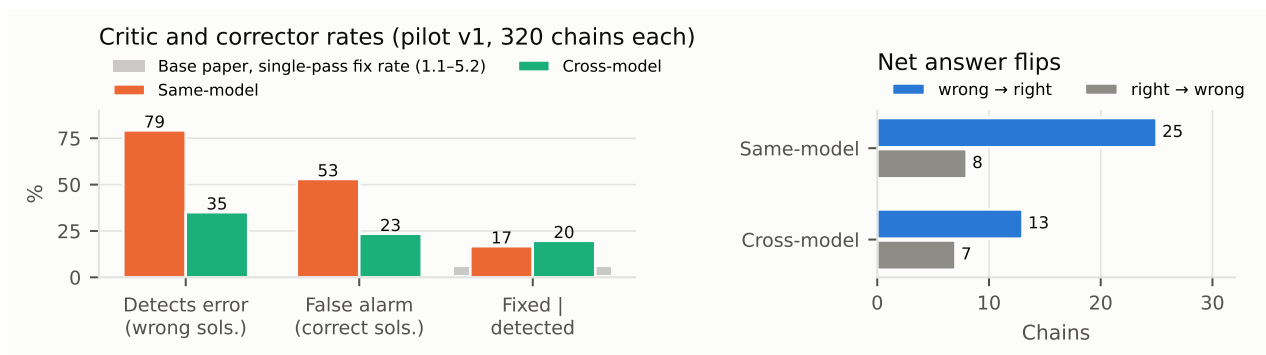


Fig. 7. Correction loop in the pilot (v1). Left: detection, false alarms and the fraction of detected wrong solutions that end correct (grey band: the base paper's single-pass 1.1–5.2%). Right: answers flipped wrong→right vs. right→wrong.

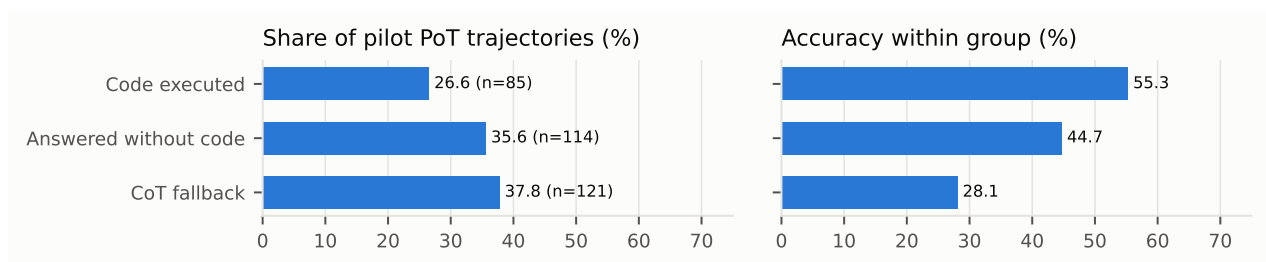


Fig. 8. Pilot PoT trajectories (320) by outcome, and accuracy within each group.

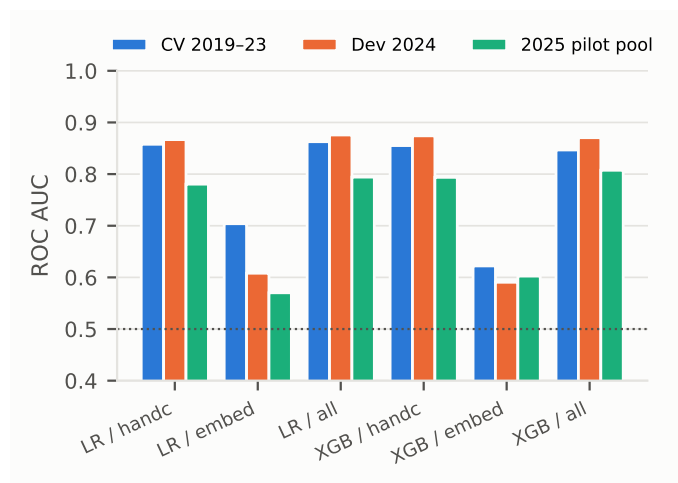


Fig. 9. Pilot verifier: ROC AUC per model and feature set (handcrafted, embedding, all).

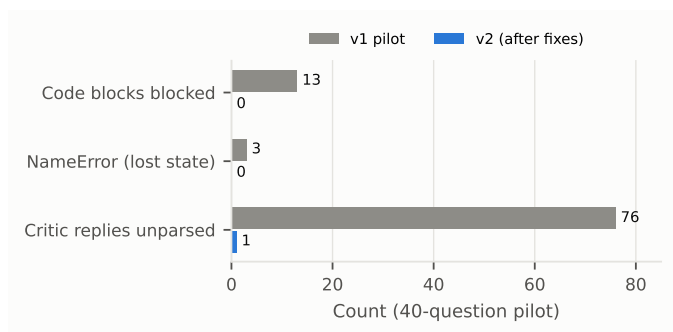


Fig. 10. Failures in the pilot before (v1) and after (v2) the fixes.

TABLE V
METHOD-LEVEL PROBLEMS FOUND IN THE PILOT TRACES: EVIDENCE, REASON FOR FIXING, AND FIX.

#	Problem	Evidence (how we found it)	Why it matters	What we did	Effect (v2)
1	Code state lost between blocks	47 <code>NameErrors</code> : every block ran in a fresh process, but the model reuses variables from its earlier blocks	Correct computations failed; the solution then fell back to mental arithmetic, the weakness PoT is meant to remove	Earlier blocks of the same solution are replayed silently (only those that pass the safety check) before the new block	0 <code>NameErrors</code>
2	Needed library blocked	86 blocked blocks: <code>scipy.optimize/integrate</code> (root finding, integrals) were not on the whitelist	The model chose the right tool and was refused	<code>scipy</code> allowed; file-I/O and parser submodules (<code>scipy.io</code> , <code>sympy.parsing</code>) stay blocked	0 blocked
3	Critic gives no verdict	76 InternVL replies unparsed; 75 of them hit the 1,536-token cap because the critic re-solved the whole problem	No verdict $\Rightarrow$ no correction, so the cross-model critic looked weak for a formatting reason	Prompt asks for a short check; <i>verdict forcing</i> : a cut-off reply gets a short greedy continuation after a decision cue	1/501 unparsed (69 forced); detection 35 $\rightarrow$ 56%
4	Many CoT fallbacks	Fallback in 35–42% of PoT runs; 480 of 539 were the model reasoning without code until the token cap (“Wait, perhaps...” loops); only 9 were code failures	Fallback answers are the least accurate and cost a second generation	A/B test on the 480: force an answer from the truncated text vs. the SOP fallback: 21.9% vs. 30.0%	Forcing <i>rejected</i> ; SOP fallback kept (negative result)
5	SC baseline under-sampled	The full framework used as many tokens as 18 CoT samples, but only 16 existed	A too-weak SC baseline would overstate our gain	CoT raised to 20 samples per question for the final run	Fair matched compute
6	Verifier $\approx$ majority vote	Dev selection 54.2% = majority vote, despite AUC 0.87	Selection is the verifier’s job; claiming a gain would be wrong	Reported honestly (<code>primary_beats_majority_on_dev</code> flag); final verifier trains on $\approx 8\times$ more data	Open; re-tested at scale
7	One prompt aborted a batch	In the v2 run a long multi-round solution gave a 12,303-token critic prompt ($>12,288$ context); vLLM’s <code>VLLMValidationError</code> is not a <code>ValueError</code> , so the whole batch aborted	One bad request could kill hours of batched GPU work	Validation errors are caught, the batch is retried request by request, and only the bad request returns an empty (conservatively handled) result	No batch aborts since

TABLE VI
VERIFICATION RUN OF THE FIXES ON THE SAME PILOT QUESTIONS.

Pilot check (40 q, cross critic)	v1	v2
Code blocks executed OK	119	138
Blocked by sandbox whitelist	13	0
<code>NameError</code> (state lost)	3	0
PoT accuracy (320 traj.)	41.2	41.2
Critic calls / unparsed	431 / 76	501 / 1
Verdicts via verdict forcing	–	69
Critic detection rate	35.1	56.4
Critic false-alarm rate	23.5	29.5
Acc. before $\rightarrow$ after	41.2 $\rightarrow$ 43.1	41.2 $\rightarrow$ 41.9

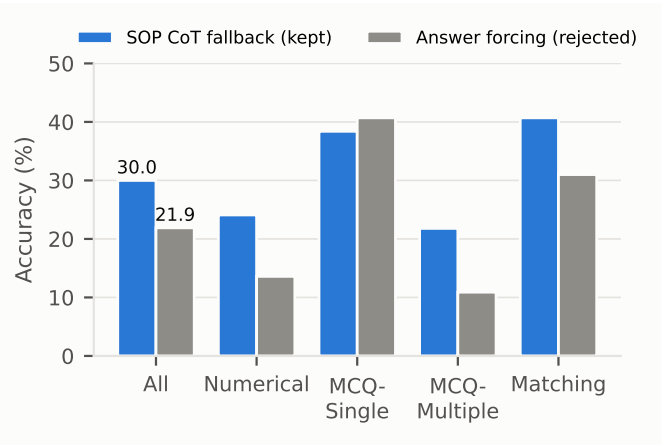


Fig. 11. A/B test of fix 4 on the 480 pilot trajectories that hit the token cap: accuracy of the SOP CoT fallback vs. forcing an answer from the truncated reasoning.

TABLE VII
BUGS FOUND BY THE PRE-RUN CODE REVIEW (THREE PARALLEL REVIEWERS), ALL FIXED BEFORE ANY GPU RUN.

#	Problem	Why it matters	What we did
8	Verifier cross-validation crashed: <code>GroupKFold</code> returns positions, pandas read them as labels	No verifier could be trained	Positional indexing (<code>iloc</code>)
9	Verifier features differed between training (pool of 8) and test (pool of 16; cumulative token counts)	Train/test mismatch silently degrades the verifier	Features computed per sub-pool; each solution stores its own token count
10	Sandbox read unbounded stdout into the parent (reproduced: 8 GB RAM)	A printing loop could crash the 15 GB lab PC	Output written to files capped by <code>RLIMIT_FSIZE</code>
11	Model ending its turn right after a code fence: code never ran	Silent loss of tool use	Detected; the block is executed and generation continues
12	No context-length guard	Long multi-turn prompts fail	Per-turn token budget and per-request retry
13	Critic-explanation regex took the first match, not the last	Wrong step passed to the corrector	Last match used
14	<code>-config</code> not passed to the subprocesses of cross-model phases	Phases could run with different settings	Passed through
15	CLs ignored the EN/HI twin correlation	Over-confident intervals (twins are near-duplicate questions)	Cluster bootstrap over twin pairs
16	Range grid of 0.01 rejected valid in-range answers (e.g. 0.275)	Correct answers scored wrong	0.001 grid; test: all 691 golds score correct
17	Duplicate question IDs could merge two questions in the KB	Wrong exemplars, possible leakage	Unique twin keys
18	Mock VLM ignored the image, which hid bug 8	Tests passed while the pipeline was broken	Image-dependent mock; end-to-end mock test now passes

TABLE VIII
INFRASTRUCTURE ISSUES ON THE LAB GPU (RTX 3060 12 GB, 15 GB RAM, SHARED PC).

#	Problem (evidence)	Why it matters	What we did
19	vLLM KV cache did not fit (0.36 GiB free after weights)	Model would not start or ran at batch size 1	Capped image pixels, <code>max_num_batched_tokens=4096</code> , FP8 KV cache: 1.9 $\times$ throughput (measured)
20	flashinfer JIT compile failed (no <code>ninja</code>)	Engine crash at start	PyTorch sampler
21	rsync exclude pattern also skipped <code>src/.../report</code>	Lab code incomplete	Anchored exclude patterns
22	CUDA out of memory during transcription at 0.92 memory utilisation	Stage crash	0.88 utilisation, 32 concurrent sequences
23	vLLM's InternVL processor rejects <code>max_dynamic_patch</code>	Critic could not load	Default tiling
24	FP8 KV cache produced garbage with Qwen2.5-VL (1.8% accuracy, junk tokens)	Reproduction would be invalid	Normal KV cache for that model; its cached outputs invalidated
25	PoT prompt produced too little code	Sandbox rarely used	Format example and a conciseness rule in the prompt
26	systemd-oomd killed the whole tmux session under memory pressure (10.5 h lost)	Silent loss of long runs	Each stage in its own scope with 3 retries; sandbox workers 8 $\rightarrow$ 4
27	oomd also kills <code>systemd-run -user</code> scopes (it watches all of the user service)	Same as 26	Jobs started with <code>nohup+setsid</code> from the ssh session, outside that policy; supervisor with retries plus a quiet local watcher

REFERENCES

- [1] A. Mukherjee and S. Ghosh, “mmJEE-Eval: A bilingual multimodal benchmark for evaluating scientific reasoning in vision-language models,” in *Proc. IJCNLP-AACL*, 2025.
- [2] X. Wang *et al.*, “Self-consistency improves chain of thought reasoning in language models,” in *Proc. ICLR*, 2023.
- [3] W. Chen *et al.*, “Program of thoughts prompting: Disentangling computation from reasoning for numerical reasoning tasks,” *TMLR*, 2023.
- [4] L. Gao *et al.*, “PAL: Program-aided language models,” in *Proc. ICML*, 2023.
- [5] A. Madaan *et al.*, “Self-Refine: Iterative refinement with self-feedback,” in *Proc. NeurIPS*, 2023.
- [6] Z. Gou *et al.*, “CRITIC: Large language models can self-correct with tool-interactive critiquing,” in *Proc. ICLR*, 2024.
- [7] J. Huang *et al.*, “Large language models cannot self-correct reasoning yet,” in *Proc. ICLR*, 2024.
- [8] K. Cobbe *et al.*, “Training verifiers to solve math word problems,” arXiv:2110.14168, 2021.
- [9] H. Lightman *et al.*, “Let’s verify step by step,” in *Proc. ICLR*, 2024.
- [10] P. Lewis *et al.*, “Retrieval-augmented generation for knowledge-intensive NLP tasks,” in *Proc. NeurIPS*, 2020.
- [11] J. Johnson, M. Douze and H. Jégou, “Billion-scale similarity search with GPUs,” *IEEE Trans. Big Data*, 2019.
- [12] S. Xiao *et al.*, “C-Pack: Packed resources for general Chinese embeddings,” in *Proc. SIGIR*, 2024.
- [13] W. Kwon *et al.*, “Efficient memory management for large language model serving with PagedAttention,” in *Proc. SOSP*, 2023.
- [14] Qwen Team, “Qwen3-VL,” model release and technical report, 2025.
- [15] OpenGVLab, “InternVL3.5,” model release and technical report, 2025.